Apostila: range() e for em Python

Algoritmos e Lógica de Programação · ETE Professor Paulo Freire · Oct 27, 2026 · @Vinicius

1. Introdução: por que repetir?

Ao final desta apostila, você será capaz de usar o laço `for` junto com a função `range()` para repetir instruções um número definido de vezes.

Objetivos da aula:

- Entender o que é uma estrutura de repetição (laço).
- Escrever laços `for` para percorrer textos e listas.
- Usar `range()` com 1, 2 e 3 argumentos.
- Resolver problemas com contadores e acumuladores.

Imagine que você precisa mostrar "Olá!" na tela 5 vezes. Sem laço, o código fica assim:

```
print("Olá!")
print("Olá!")
print("Olá!")
print("Olá!")
print("Olá!")
```

E se fossem 1.000 vezes? Copiar e colar não resolve. Para isso existem as **estruturas de repetição**: elas mandam o computador executar o mesmo bloco de código várias vezes. Com `for` e `range()`, o mesmo programa vira:

```
for i in range(5):
    print("Olá!")
```

Duas linhas fazem o trabalho de cinco — ou de mil, trocando apenas o número.

2. O laço for

O `for` percorre uma sequência, um item de cada vez, e executa o bloco de código para cada item.

Sintaxe geral:

```
for variavel in sequencia:
    # bloco que se repete (com 4 espaços de recuo)
    instrucao
```

Partes do comando:

- `for` e `in` são palavras reservadas do Python.
- `variavel` recebe, a cada volta, o próximo item da sequência. Você escolhe o nome.
- `sequencia` pode ser um texto (string), uma lista ou um `range()`.
- Os **dois-pontos** `:` no final da linha são obrigatórios.
- A **indentação** (recuo de 4 espaços) define o que está dentro do laço.

Cada repetição do laço é chamada de **iteração**.

Percorrendo um texto

```
palavra = "ETE"

for letra in palavra:   # letra vale "E", depois "T", depois "E"
    print(letra)
```

Saída:

```
E
T
E
```

Percorrendo uma lista

```
frutas = ["maçã", "banana", "uva"]

for fruta in frutas:
    print("Eu gosto de", fruta)

print("Fim!")   # fora do laço: executa uma vez só
```

Saída:

```
Eu gosto de maçã
Eu gosto de banana
Eu gosto de uva
Fim!
```

Observe que `print("Fim!")` está sem recuo. Por isso ele não faz parte do laço e roda apenas uma vez, depois que o `for` termina.

3. A função range()

A função `range()` gera uma sequência de números inteiros. Ela é a parceira ideal do `for` quando queremos repetir algo um número exato de vezes.

Regra de ouro: o número final **nunca** entra na sequência. `range(5)` vai de 0 até 4.

range(fim) — um argumento

Começa em 0 e para **antes** de `fim`.

```
for i in range(5):
    print(i)
# Saída: 0 1 2 3 4  (cinco números, começando do zero)
```

range(início, fim) — dois argumentos

Começa em `início` e para **antes** de `fim`.

```
for i in range(1, 6):
    print(i)
# Saída: 1 2 3 4 5
```

Dica: para contar de 1 até N, use `range(1, N + 1)`.

range(início, fim, passo) — três argumentos

O `passo` diz de quanto em quanto a sequência anda.

```
for i in range(0, 11, 2):
    print(i)
# Saída: 0 2 4 6 8 10  (de 2 em 2)
```

Com passo negativo, a contagem é **regressiva**:

```
for i in range(10, 0, -1):
    print(i)
print("Feliz Ano Novo!")
# Saída: 10 9 8 7 6 5 4 3 2 1 Feliz Ano Novo!
```

Tabela comparativa

Chamada	Início	Fim (não incluso)	Passo	Números gerados
`range(5)`	0	5	1	0, 1, 2, 3, 4
`range(1, 6)`	1	6	1	1, 2, 3, 4, 5
`range(3, 8)`	3	8	1	3, 4, 5, 6, 7
`range(0, 11, 2)`	0	11	2	0, 2, 4, 6, 8, 10
`range(1, 10, 3)`	1	10	3	1, 4, 7
`range(5, 0, -1)`	5	0	-1	5, 4, 3, 2, 1
`range(5, 1)`	5	1	1	(vazio — nada é gerado)

Para ver os números de um `range()` sem usar laço, converta para lista:

```
print(list(range(1, 10, 3)))   # [1, 4, 7]
```

4. Combinando for e range(): exemplos comentados

Os exemplos abaixo mostram os usos mais comuns do `for` com `range()` em problemas de lógica.

Exemplo 1 — Repetir uma ação N vezes

```
n = int(input("Quantas vezes? "))

for i in range(n):
    print("Estudando Python!")
```

Aqui a variável `i` nem é usada dentro do laço; ela só conta as voltas. Nesses casos, é comum escrever `for _ in range(n):`.

Exemplo 2 — Tabuada

```
numero = int(input("Digite um número: "))

for i in range(1, 11):              # i vai de 1 até 10
    resultado = numero * i
    print(numero, "x", i, "=", resultado)
```

Saída para o número 7:

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Exemplo 3 — Acumulador: soma de 1 até 100

Um **acumulador** é uma variável que guarda um total que cresce a cada volta. Ela deve ser iniciada **antes** do laço.

```
soma = 0                    # 1. inicia o acumulador fora do laço

for i in range(1, 101):     # 2. i vai de 1 até 100
    soma = soma + i         # 3. acrescenta i ao total (ou: soma += i)

print("Soma:", soma)        # 4. mostra o resultado depois do laço
# Saída: Soma: 5050
```

Teste de mesa

Acompanhar as variáveis volta a volta ajuda a entender o laço. Veja as primeiras iterações do Exemplo 3:

Iteração	Valor de i	soma antes	soma depois
1ª	1	0	1
2ª	2	1	3
3ª	3	3	6
4ª	4	6	10
…	…	…	…
100ª	100	4950	5050

Exemplo 4 — Contador: quantos números pares?

Um **contador** soma 1 sempre que uma condição é verdadeira.

```
pares = 0

for i in range(1, 21):
    if i % 2 == 0:           # % é o resto da divisão; resto 0 = par
        pares = pares + 1

print("Pares entre 1 e 20:", pares)
# Saída: Pares entre 1 e 20: 10
```

Exemplo 5 — Lendo vários valores do usuário

```
total = 0

for i in range(1, 4):
    nota = float(input(f"Digite a nota {i}: "))
    total = total + nota

media = total / 3
print(f"Média: {media:.1f}")
```

O laço pede 3 notas, acumula a soma e, ao final, calcula a média.

5. Erros comuns

A maioria dos erros com `for` e `range()` vem de esquecer os dois-pontos, errar o recuo ou esperar que o número final seja incluído.

Erro	Código errado	Código correto	O que acontece
Esquecer os dois-pontos	`for i in range(5)`	`for i in range(5):`	`SyntaxError`
Esquecer o recuo	`print(i)` sem espaços abaixo do `for`	`print(i)` com 4 espaços	`IndentationError`
Achar que o fim entra	`range(1, 10)` para contar até 10	`range(1, 11)`	Para no 9
Usar número decimal	`range(0, 1, 0.1)`	Use apenas inteiros	`TypeError`
Passo zero	`range(0, 10, 0)`	Passo diferente de zero	`ValueError`

Erro	Código errado	Código correto	O que acontece
Passo errado na contagem regressiva	`range(10, 0)`	`range(10, 0, -1)`	Laço não executa
Iniciar acumulador dentro do laço	`soma = 0` dentro do `for`	`soma = 0` antes do `for`	A soma zera a cada volta
Usar texto do input direto	`range(input("N: "))`	`range(int(input("N: ")))`	`TypeError`

6. Exercícios de fixação

Resolva os exercícios em ordem; eles ficam mais difíceis a cada bloco.

Parte A — Qual é a saída?

Sem executar, escreva os números que cada `range()` gera. Depois confira com `print(list(...))`.

1. `range(4)`
2. `range(2, 7)`
3. `range(1, 20, 5)`
4. `range(10, 0, -2)`
5. `range(3, 3)`

Parte B — Complete o código

6. Complete para mostrar os números de 1 a 10:

```
for i in range(___, ___):
    print(i)
```

7. Complete para mostrar os múltiplos de 5 de 0 a 50:

```
for i in range(0, ___, ___):
    print(i)
```

Parte C — Programas

8. Escreva um programa que mostre seu nome 10 vezes.

9. Peça um número N ao usuário e mostre todos os números de 1 até N.
10. Mostre a contagem regressiva de 20 até 0 e, ao final, a frase "Lançar!".
11. Mostre todos os números ímpares entre 1 e 50.
12. Calcule e mostre a soma dos números de 1 até N, com N informado pelo usuário.
13. Peça 5 números ao usuário e mostre a soma e a média deles.
14. Conte quantos números entre 1 e 100 são divisíveis por 3 **e** por 5 ao mesmo tempo.

Desafio

15. **Fatorial.** O fatorial de N (escrito N!) é o produto de 1 até N. Exemplo: 5! = 1 × 2 × 3 × 4 × 5 = 120. Escreva um programa que leia N e mostre o seu fatorial. Dica: o acumulador de um produto começa em 1, não em 0.
16. **Escadinha.** Leia um número N e desenhe uma escada de asteriscos. Para N = 4:

```
*
**
***
****
```

Dica: em Python, `"*" * 3` resulta em `"***"`.

7. Resumo rápido

O `for` repete um bloco para cada item de uma sequência, e o `range()` gera essa sequência de números.

- `for variavel in sequencia:` — não esqueça os dois-pontos e o recuo de 4 espaços.
- `range(fim)` — de 0 até fim − 1.
- `range(início, fim)` — de início até fim − 1.
- `range(início, fim, passo)` — anda de passo em passo; passo negativo conta para trás.
- O valor de `fim` **nunca** é incluído.
- Para contar de 1 até N: `range(1, N + 1)`.
- **Contador:** começa em 0 e soma 1 quando a condição é verdadeira.
- **Acumulador:** começa antes do laço (0 para soma, 1 para produto) e é atualizado a cada volta.
- Para ver os números gerados: `print(list(range(...)))`.